Design Patterns 2: Iterator, Factory, Observer, Publisher-Subscriber, Repository, Entities

Alexandru Olteanu

Universitatea Politehnica Bucuresti
Facultatea de Automatică si Calculatoare, Departamentul Calculatoare
alexandru.olteanu@cs.pub.ro

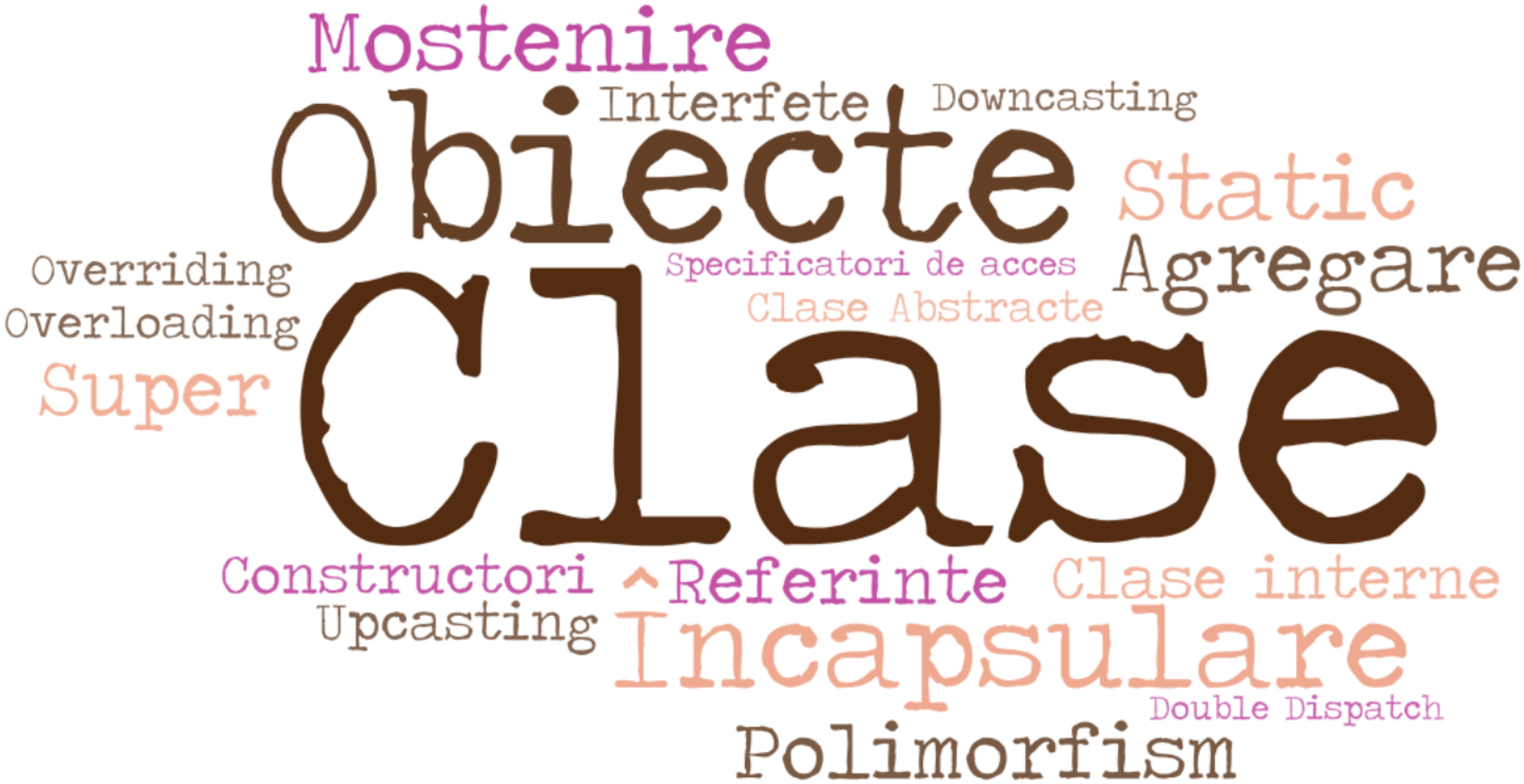
OOP, 2018



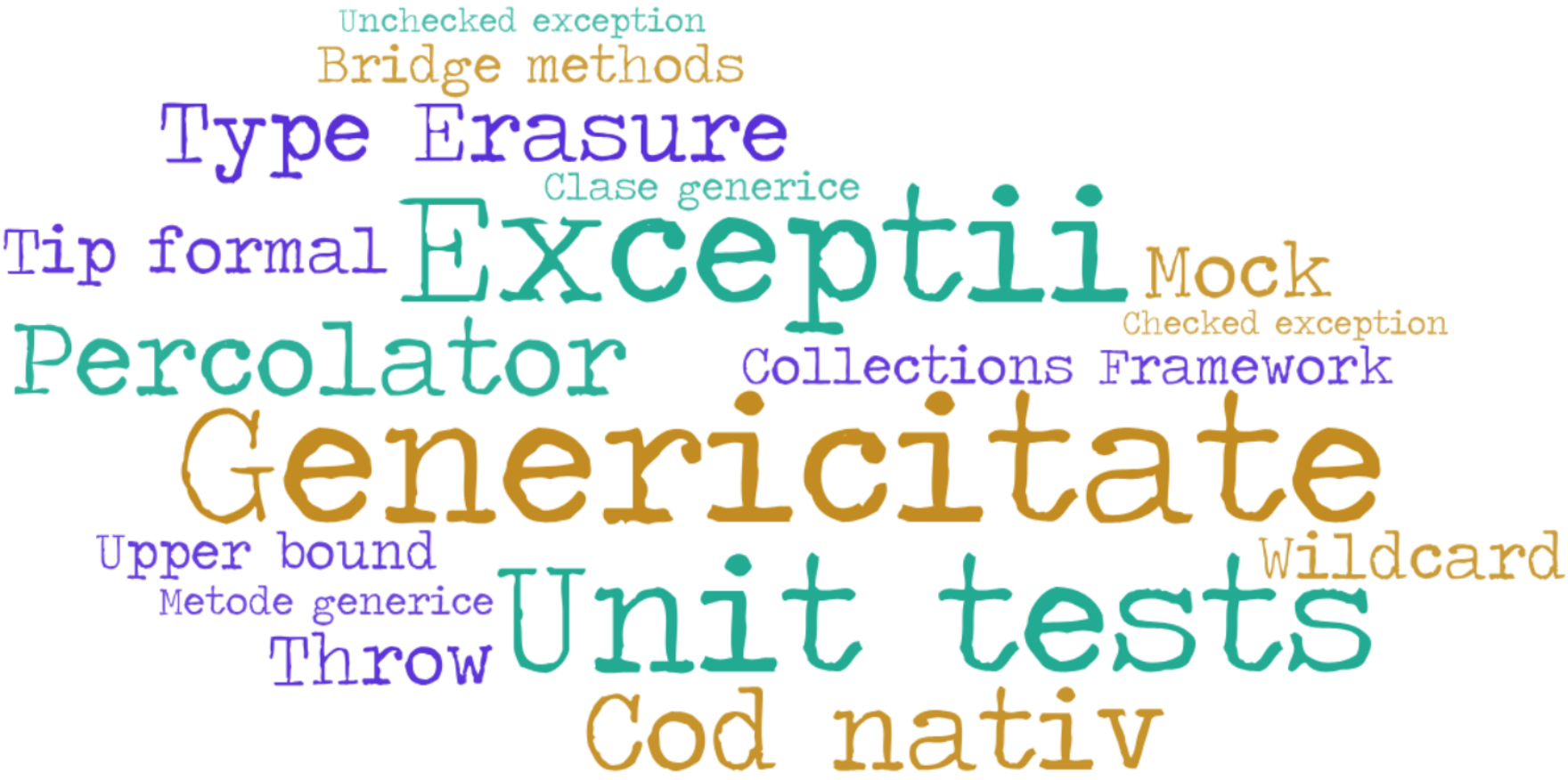




Partea 1: Bazele teoretice ale OOP



Partea a 2-a: Antract pragmatic



Curs 09: Design Patterns 1

Clasificare Design Patterns:

- *creational patterns*: Singleton
- *structural patterns*: Adapter
- *behavioral patterns*: Visitor
- *architectural patterns*: Client-Server, Three-Tier

Frameworks - Inversion of Control implementation:

- Dependency Injection
- Strategy pattern

Iterator

Description

Iterator provides a way to access the elements of an aggregate object (e.g. list etc.) sequentially without exposing its underlying representation

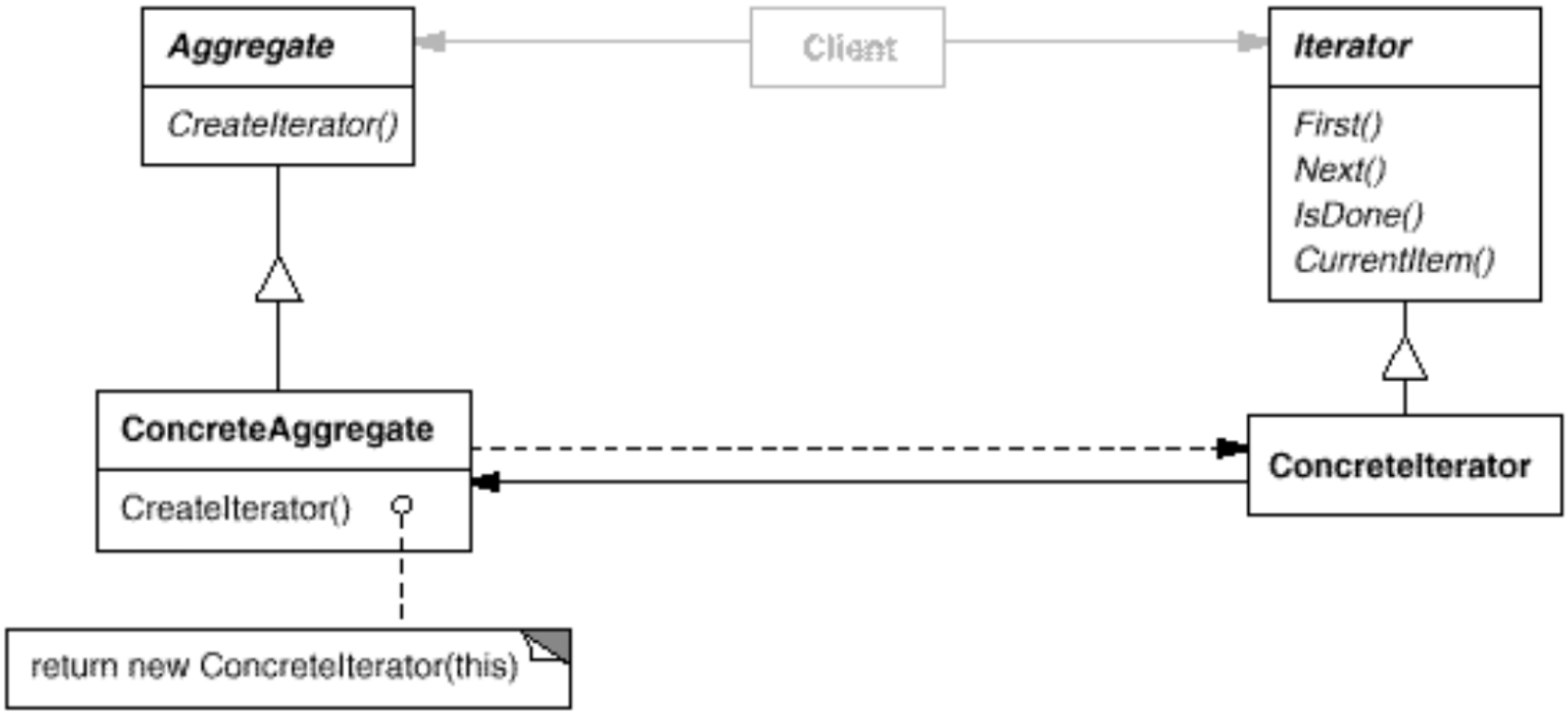
Problem that the pattern solves

- de multe ori avem nevoie sa parcurgem elementele unor structuri de date agregate (liste, arbori, grafuri etc.)

Problem that the pattern solves

- de multe ori avem nevoie sa parcurgem elementele unor structuri de date agregate (liste, arbori, grafuri etc.)
- parcurgerea se poate face in multe feluri (listele de la stanga la dreapta, de la dreapta la stanga, cu filtru, pozitii pare etc.), implementarea tuturor ar incarca structura de date agregata in mod inutil

Solution Structure



* E. Gamma, R. Helm, R. Johnson, J. Vlissides "Design Patterns: Elements of Reusable Object-Oriented Software"

Solution Details

- crearea iteratorului ar trebui sa apartina structurii de date agregate, care isi cunoaste implementarea

Solution Details

- crearea iteratorului ar trebui sa apartina structurii de date agregate, care isi cunoaste implementarea
- un iterator robust suporta adaugari si extrageri de elemente pe parcursul traversarii

Implementation Examples

```
for (Iterator<E> iter = list.iterator(); iter.hasNext(); ) {
    E element = iter.next();
    // 1 - can call methods of element
    // 2 - can use iter.remove() to remove
    //       the current element from the list

    // ...
}
```

Implementation Examples

```
for
(ListIterator<E> iter=list.listIterator();iter.hasNext();){
    E element = iter.next();
    // 1 - can call methods of element
    // 2 - can use iter.remove() to remove
    //       the current element from the list
    // 3 - can use iter.add(...) to insert
    //       a new element into the list
    //       between element and iter->next()
    // 4 - can use iter.set(...) to replace
    //       the current element

    // ...
}
```

Implementation Examples

```
for (E element : list) {
    // 1 - can call methods of element

    // ...
}
```

Aceasta implementare ascunde un iterator, asta insa face ca iteratorul sa nu poata fi folosit pentru adaugare si extragere de elemente

Exercitii

- FilterIterator: http://codereview.stackexchange.com/questions/112109/filtered-iterator
- Filter cu Stream din Java 1.8

Consequences

- ofera posibilitatea de a varia simplu modalitatea de traversare

Consequences

- ofera posibilitatea de a varia simplu modalitatea de traversare
- simplifica interfata structurii de date agregate

Consequences

- ofera posibilitatea de a varia simplu modalitatea de traversare
- simplifica interfata structurii de date agregate
- pot exista mai multi iteratori concomitent asupra unei structuri de date agregate, deci mai multe traversari concomitente

Factory

Description

Abstract Factory provides an interface for creating families of related or dependent objects without specifying their concrete classes.
Factory Method defines an interface for creating an object, but lets subclasses decide which class to instantiate.

Description

Abstract Factory provides an interface for creating families of related or dependent objects without specifying their concrete classes.
Factory Method defines an interface for creating an object, but lets subclasses decide which class to instantiate.

AbstractFactory classes are often implemented with Factory Methods, but they can also be implemented using Prototype. A concrete factory is often a Singleton.

Description

	AbstractFactory	Builder
what?	instances of related classes	complex object, step by step
object available	immediately	as a last step

Problem that the pattern solves

- vrem sa generam instante de clase inrudite (mostenesc o interfata comuna sau extind o clasa abstracta comuna)

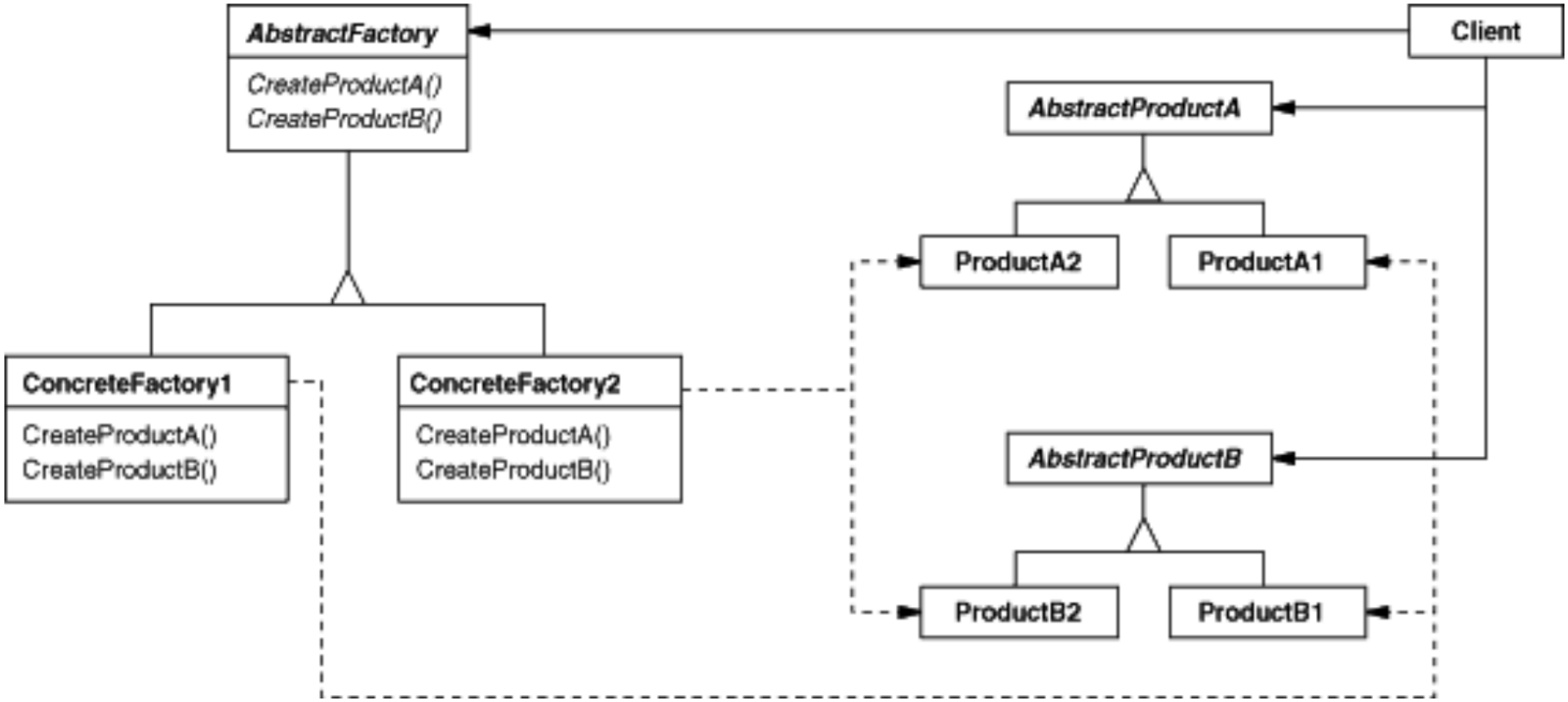
Problem that the pattern solves

- vrem sa generam instante de clase inrudite (mostenesc o interfata comuna sau extind o clasa abstracta comuna)
- clasa care solicita instantele nu are nevoie sa stie toate implementarile posibile, ci doar caracteristicile obiectului ce trebuie implementat

Problem that the pattern solves

- vrem sa generam instante de clase inrudite (mostenesc o interfata comuna sau extind o clasa abstracta comuna)
- clasa care solicita instantele nu are nevoie sa stie toate implementarile posibile, ci doar caracteristicile obiectului ce trebuie implementat
- vrem sa constrangem clientul sa foloseasca obiectele inrudite impreuna sau sa ii ascundem implementarile

Solution Structure



* E. Gamma, R. Helm, R. Johnson, J. Vlissides "Design Patterns: Elements of Reusable Object-Oriented Software"

- Products like soups, salads, main dishes etc.
- Factories like ItalianRestaurant, AsianRestaurant etc.

Solution Details

- de cele mai multe ori este suficienta o singura ConcreteFactory (dar sunt situatii in care putem avea mai multe)

Solution Details

- de cele mai multe ori este suficienta o singura ConcreteFactory (dar sunt situatii in care putem avea mai multe)
- crearea concreta a obiectelor apartine ConcreteFactory

Implementation Examples

Urmariti acest ▸ exemplu de Factory in C#

Exercitii

- Cautati care sunt clasele Factory in ▸ Java API
- Urmariti ▸ folosirea StringBuilder si sesizati diferentele dintre Factory si Builder
- Ar trebui sa puteti intelege tot ce scrie in ▸ acest articol
- Ce e prea mult strica: ▸ articol foarte interesant de la unul dintre fondatorii StackOverflow

Consequences

- izoleaza clientul de implementarea concreta a claselor

Consequences

- izoleaza clientul de implementarea concreta a claselor
- se poate schimba familia de produse usor (prin inlocuirea cu un alt Factory), dar adaugarea unui nou tip de produs este mai dificila din perspectiva clientului

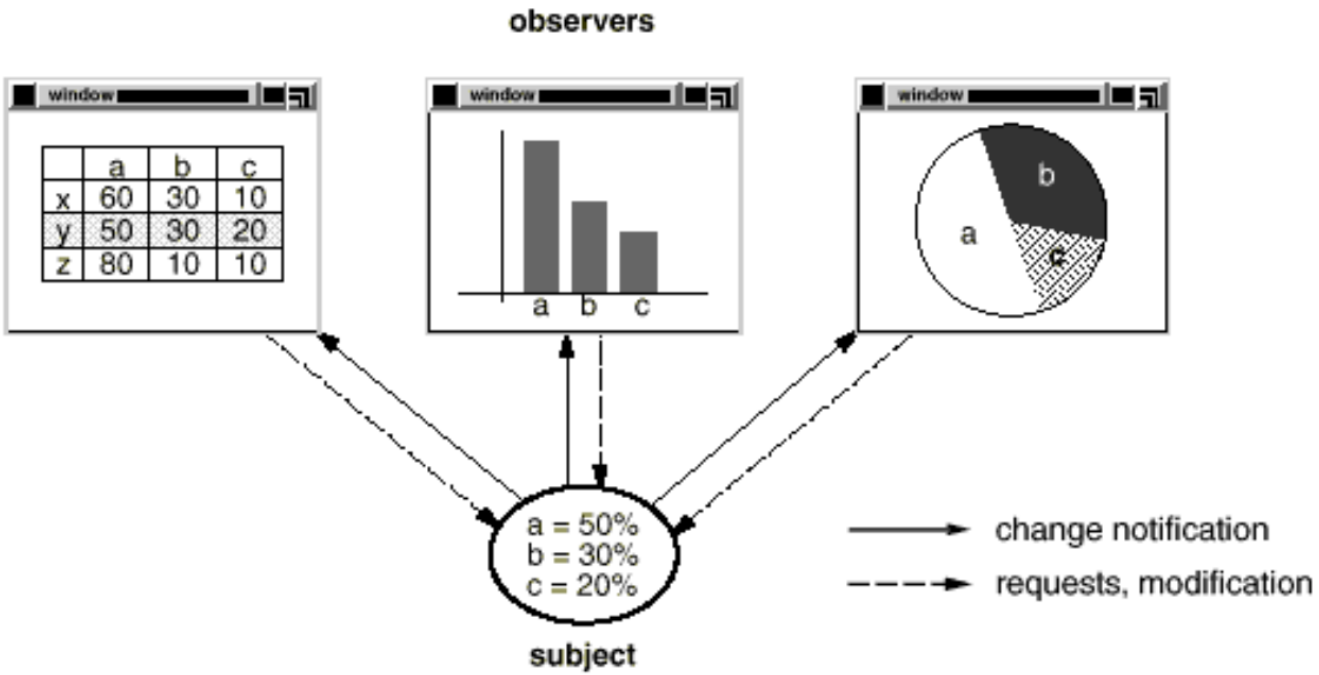
Consequences

- izoleaza clientul de implementarea concreta a claselor
- se poate schimba familia de produse usor (prin inlocuirea cu un alt Factory), dar adaugarea unui nou tip de produs este mai dificila din perspectiva clientului
- promoveaza consistenta intre produse

Observer

Description

Observer defines a dependency between objects so that when one object changes state, all its dependents are notified and updated automatically (aka Publisher-Subscriber)



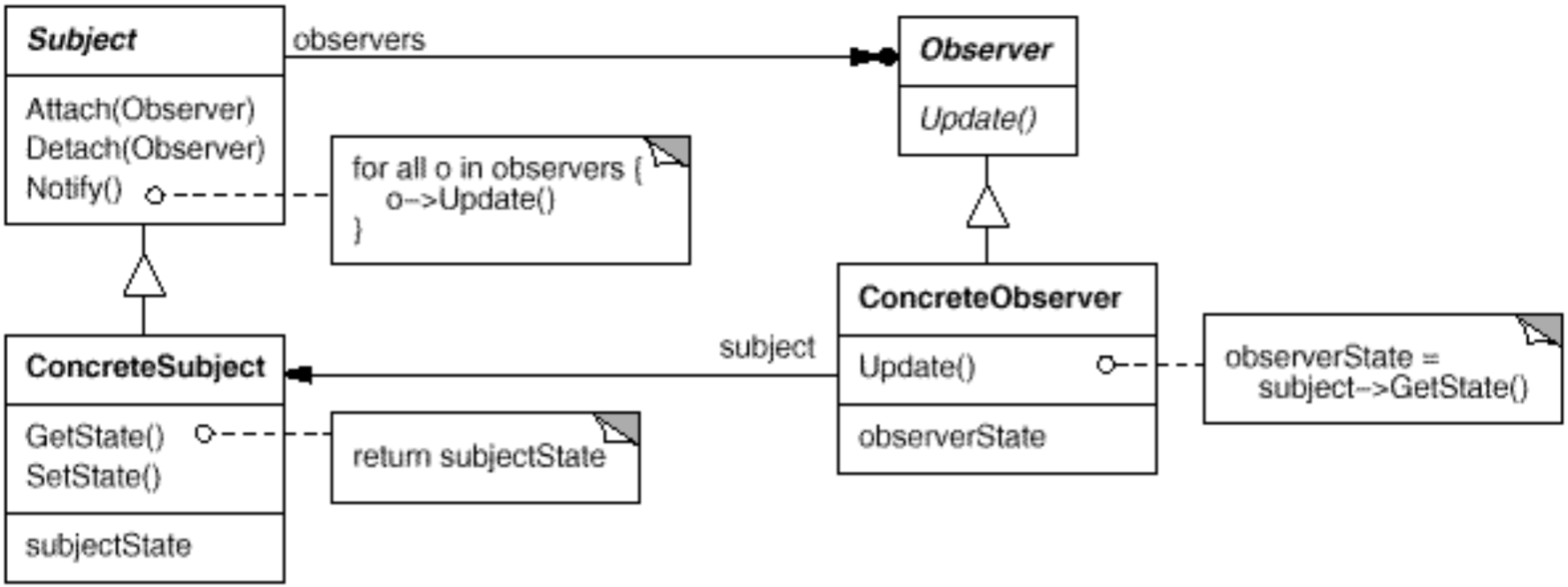
Problem that the pattern solves

- mai multe obiecte depind de aceeasi informatie, si

Problem that the pattern solves

- mai multe obiecte depind de aceeasi informatie, si
- nu vrem sa le legam strans, vrem ca relatia dintre ele sa poata fi dinamica

Solution Structure



* E. Gamma, R. Helm, R. Johnson, J. Vlissides "Design Patterns: Elements of Reusable Object-Oriented Software"

Solution Details

- Subject isi cunoaste observatorii si ofera mecanism de abonare si dezabonare; pot fi oricati observatori

Solution Details

- Subject isi cunoaste observatorii si ofera mecanism de abonare si dezabonare; pot fi oricati observatori
- ConcreteSubject notifica toti observatorii cand isi schimba starea

Solution Details

- Subject isi cunoaste observatorii si ofera mecanism de abonare si dezabonare; pot fi oricati observatori
- ConcreteSubject notifica toti observatorii cand isi schimba starea
- ConcreteObserver mentine o referinta la ConcreteSubject si implementeaza Update pentru a isi mentine starea in conformitate cu starea lui ConcreteSubject

Implementation Examples

Remember ▸ Lab 5: Clase Interne

Exercitii

Ganditi-va cum poate fi folosit pattern-ul observer pentru a implementa o aplicatie:

- de tip Internet of Things: server care primeste date de la mai multi senzori
- pentru chat

Consequences

- relatia dintre observatori si obiectele observate este abstracta, fiecare stie despre celelalt interfata / clasa abstracta pe care o decupleaza

Consequences

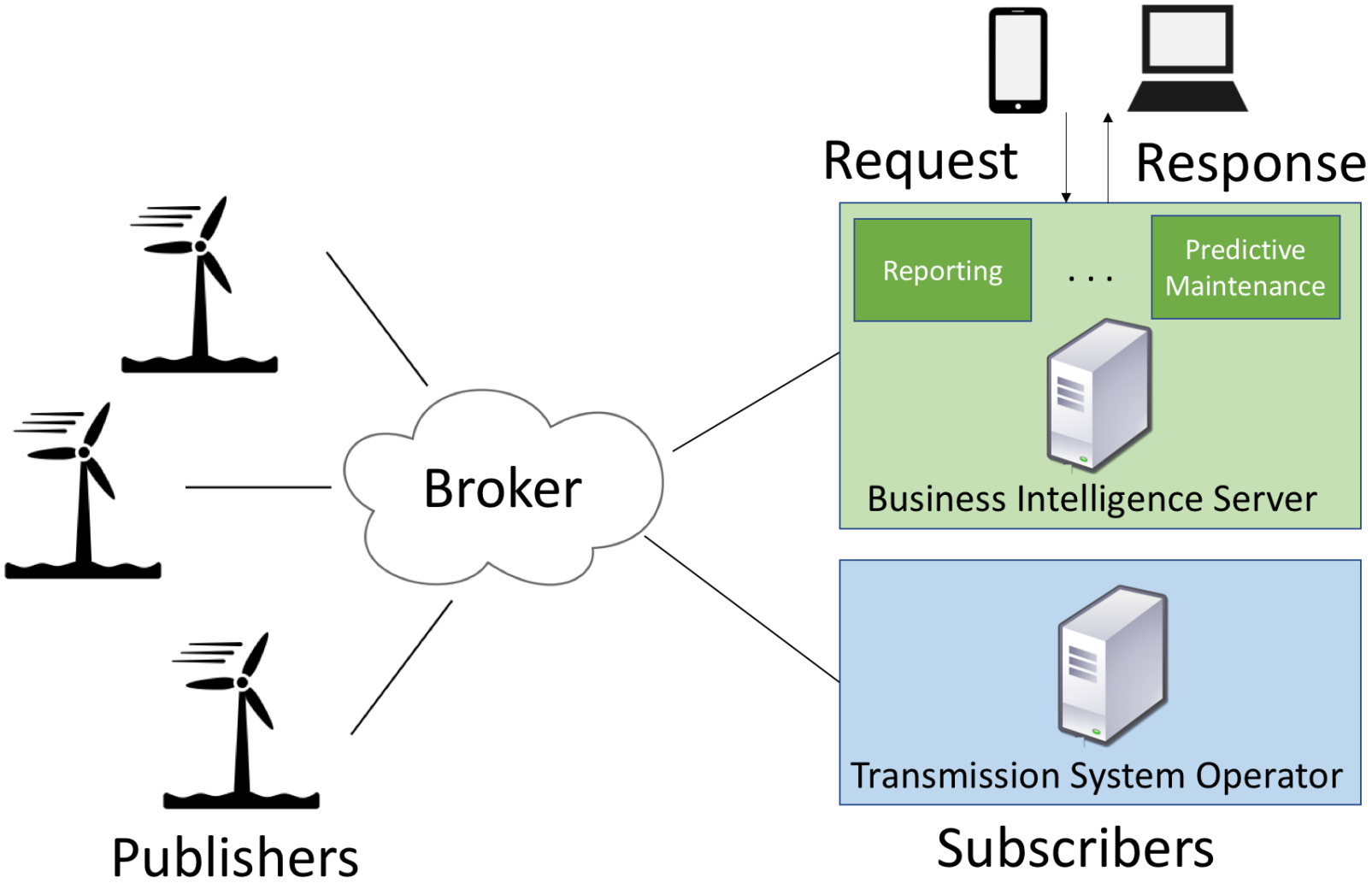
- relatia dintre observatori si obiectele observate este abstracta, fiecare stie despre celelalt interfata / clasa abstracta pe care o decupleaza
- difuzarea larga a mesajului (broadcast)

Consequences

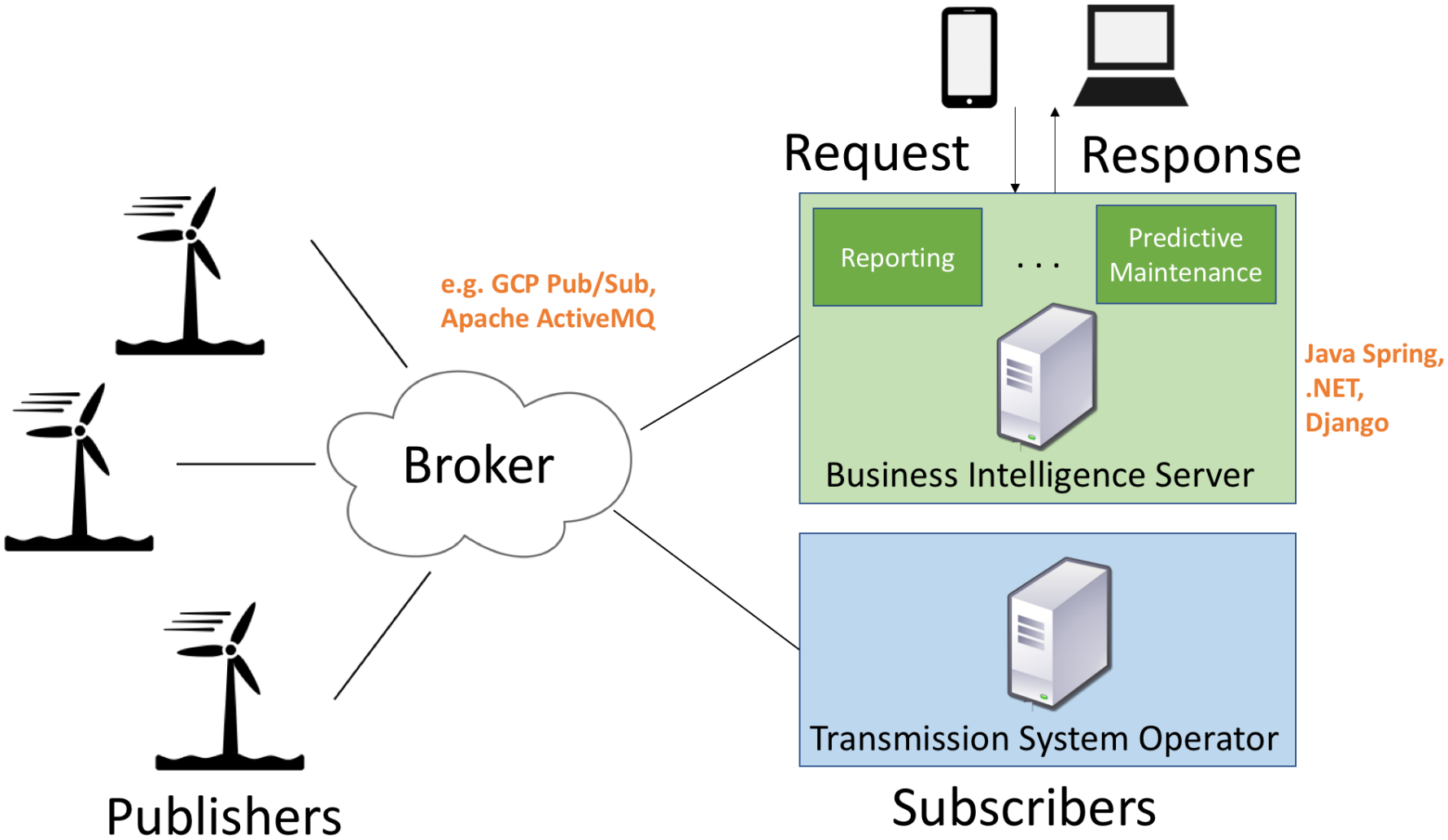
- relatia dintre observatori si obiectele observate este abstracta, fiecare stie despre celelalt interfata / clasa abstracta pe care o decupleaza
- difuzarea larga a mesajului (broadcast)
- in forma de baza, obiectele observate nu anunta ce anume s-a schimbat; se pot imagina scenarii in care o dezvoltare detaliata a unui astfel de protocol sa fie necesara

Architectural Patterns

Publisher-Subscriber vs Request-Response

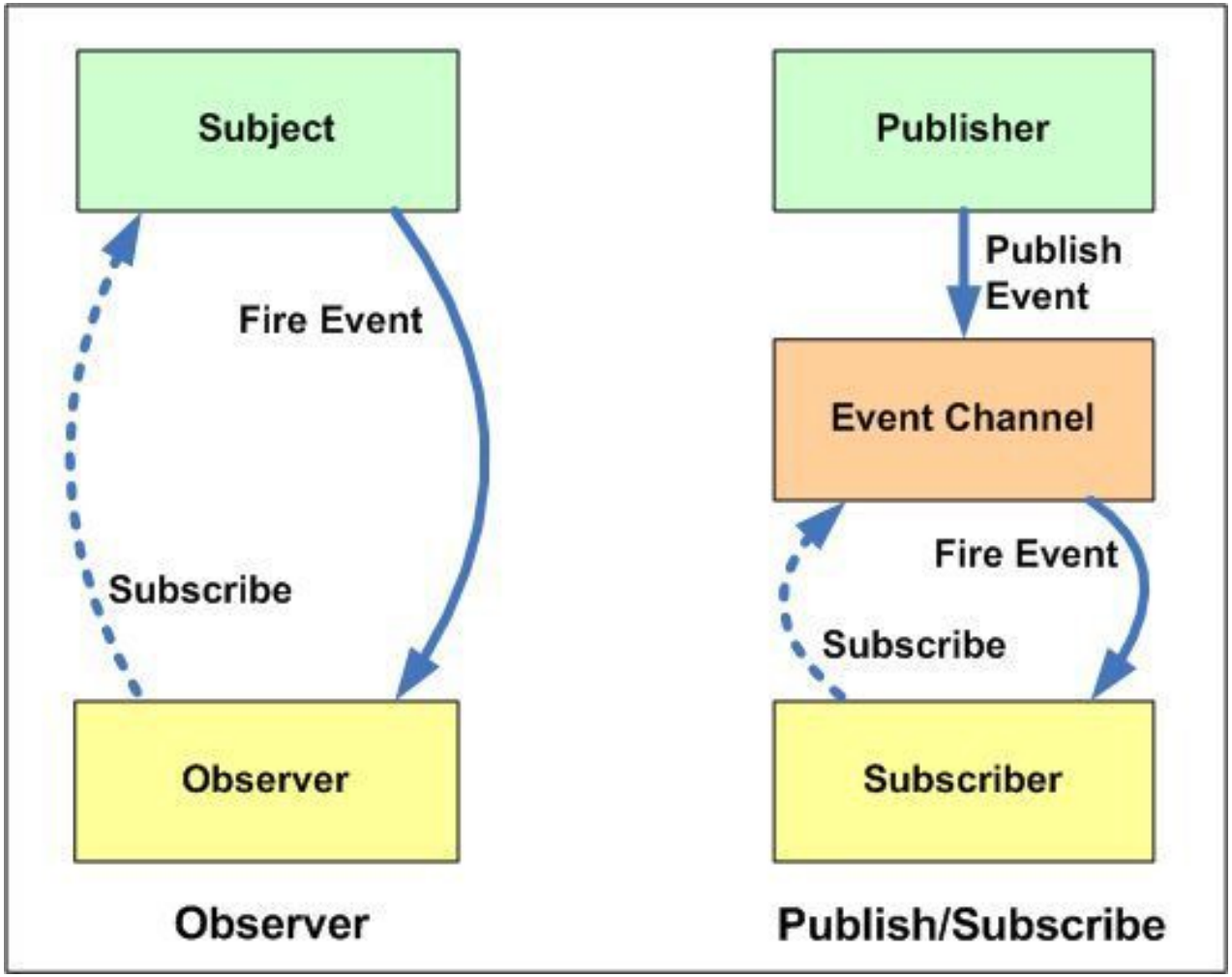


Publisher-Subscriber vs Request-Response



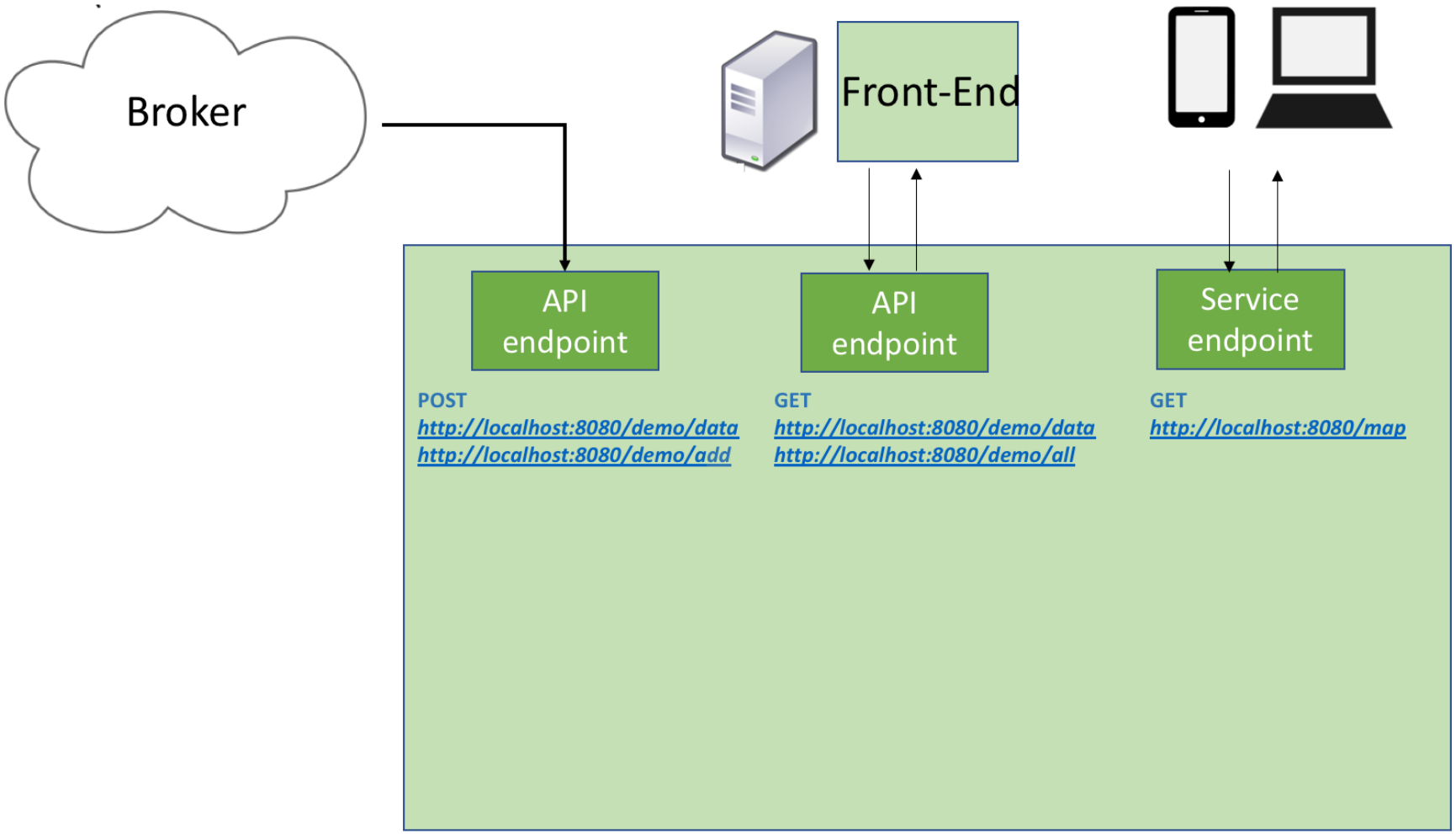
Publisher-Subscriber vs Observer

Publisher-Subscriber vs Observer



▸ Observer vs Pub-Sub pattern

Zoom in on server: endpoints



CRUD. REST.

Definition

Create, Read, Update, and Delete (CRUD) are the four basic functions implemented in relational database applications.

Operation	**RESTful WS via HTTP**
Create	POST
Read	GET
Update	PUT
Delete	DELETE

Definition

Representational State Transfer (REST) is a software architectural style used for creating web services. RESTful web services allow the requesting systems to interact with web resources using a set of stateless operations, based on CRUD.

MVC

Model-View-Controller (MVC) is an architectural design pattern that deals with separation of concerns, splitting the code in three components:

- Model: data, state, business logic
- View: representation of data (usually UI, but nowadays JSON/XML may be interpreted as views)
- Controller: the logic for reacting to user interaction and model changes

Tweaked over the years to accommodate various technologies, improve testability etc.

Pentru aprofundarea subiectului

- MVC vs. MVP vs. MVVM, Niraj Bhatt
- MVC vs. MVP vs. MVVM on Android, Eric Maxwell

MVC vs Front Controller

Front Controller is a twist on a typical MVC designed for large web applications: there is a main controller that dispatches actions on various controllers:

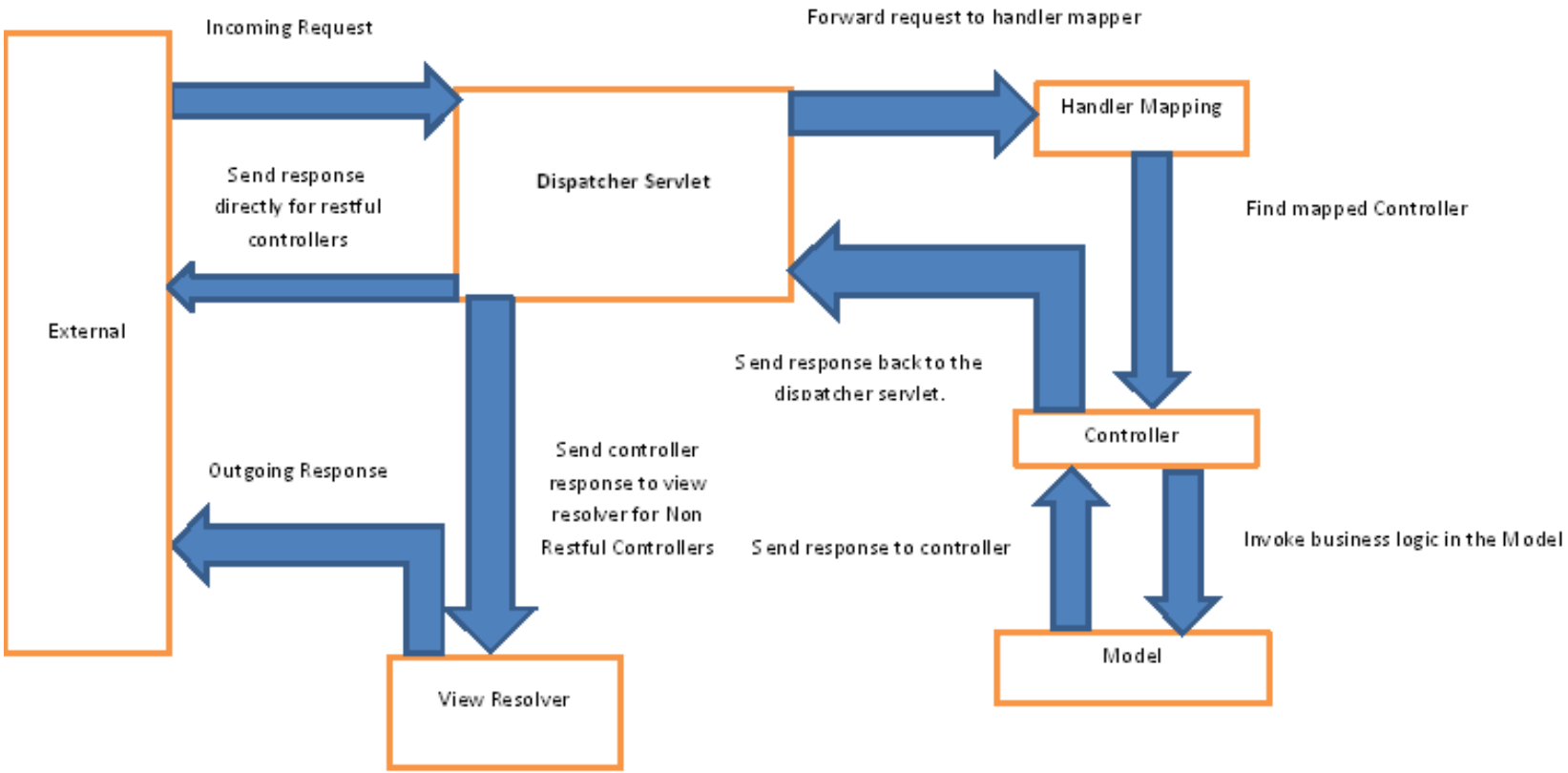
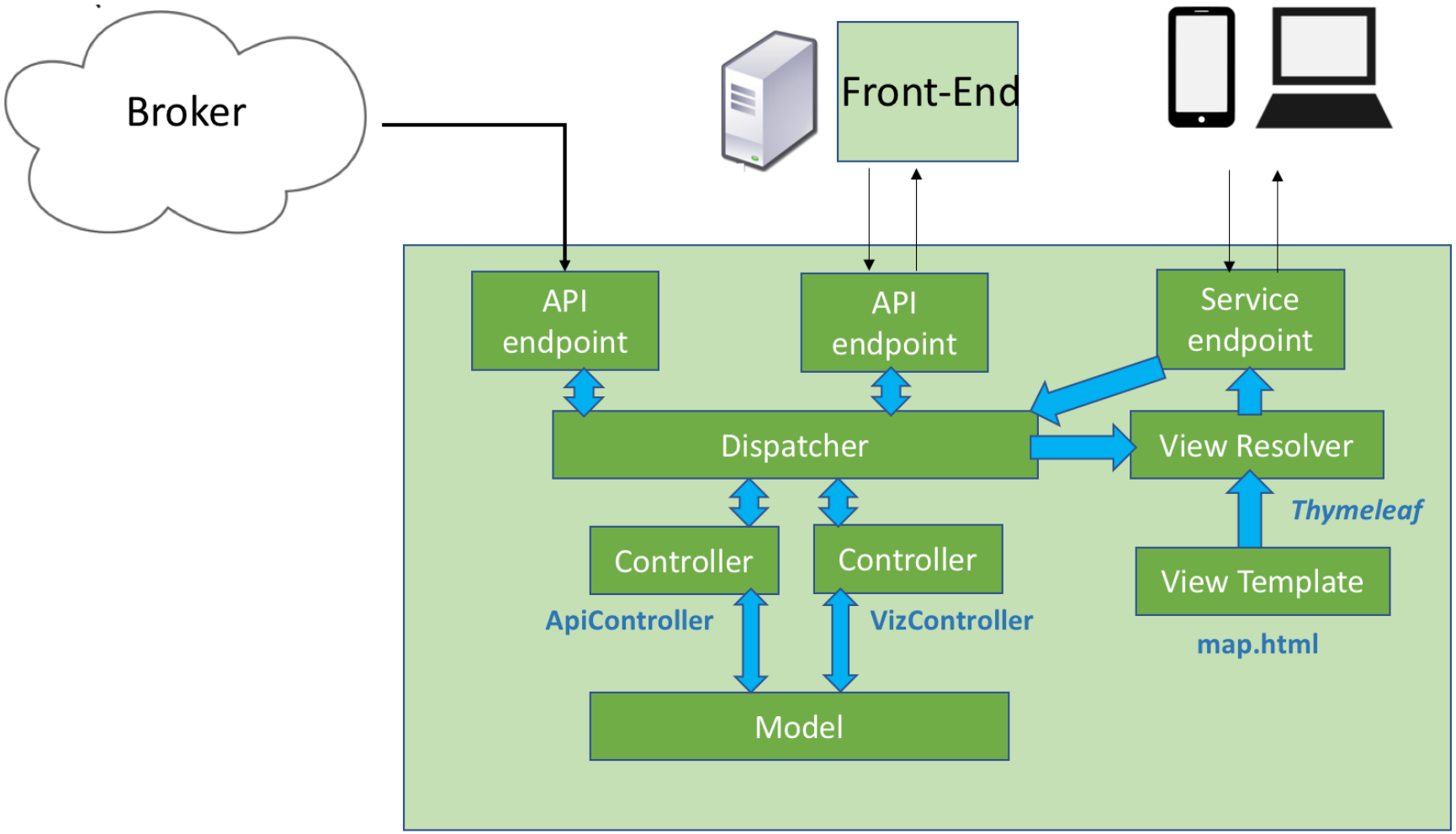


Fig 1 MVC Architecture flow

Zoom in on server: view and controller



POJO

Definition

Plain Old Java Object (POJO) is an ordinary Java object, not bound by any special restriction and not requiring any class path.

Reaction to Enterprise Java Beans (EJB): "We wondered why people were so against using regular objects in their systems and concluded that it was because simple objects lacked a fancy name. So we gave them one, and it's caught on very nicely." - Martin Fowler, Rebecca Parsons, Josh MacKenzie

Java Beans

Definition

Java Beans are classes that encapsulate many objects into a single object (the bean), to serve as reusable software components for Java.

Java Beans:

- are serializable
- have a public default constructor
- allow access to properties using getter and setter methods

Java Beans instances are POJOs, not to be confused with Enterprise Java Beans (EJB).

Entities

Definition

Java Persistence API (JPA) Entities are classes specifically designated by the programmer whose nontransient fields should be persisted to a relational database using the services of an entity manager obtained from a JPA persistence provider.

Entities instances are POJOs.

DTOs

Definition

Data Transfer Objects (DTO) is a very simple object meant to carry data between processes, without any behavior (except for serialization, storage and retrieval).

Somewhat similar to struct in C.

Repository pattern

The Repository design pattern provides an abstraction of data, so that your application can work with a simple abstraction that has an interface approximating that of a collection.

Adding, removing, updating, and selecting items from this collection is done through a series of straightforward methods, without the need to deal with database concerns like connections, commands, cursors, or readers.

▸ Repository Pattern - A data persistence abstraction

Repository implementation

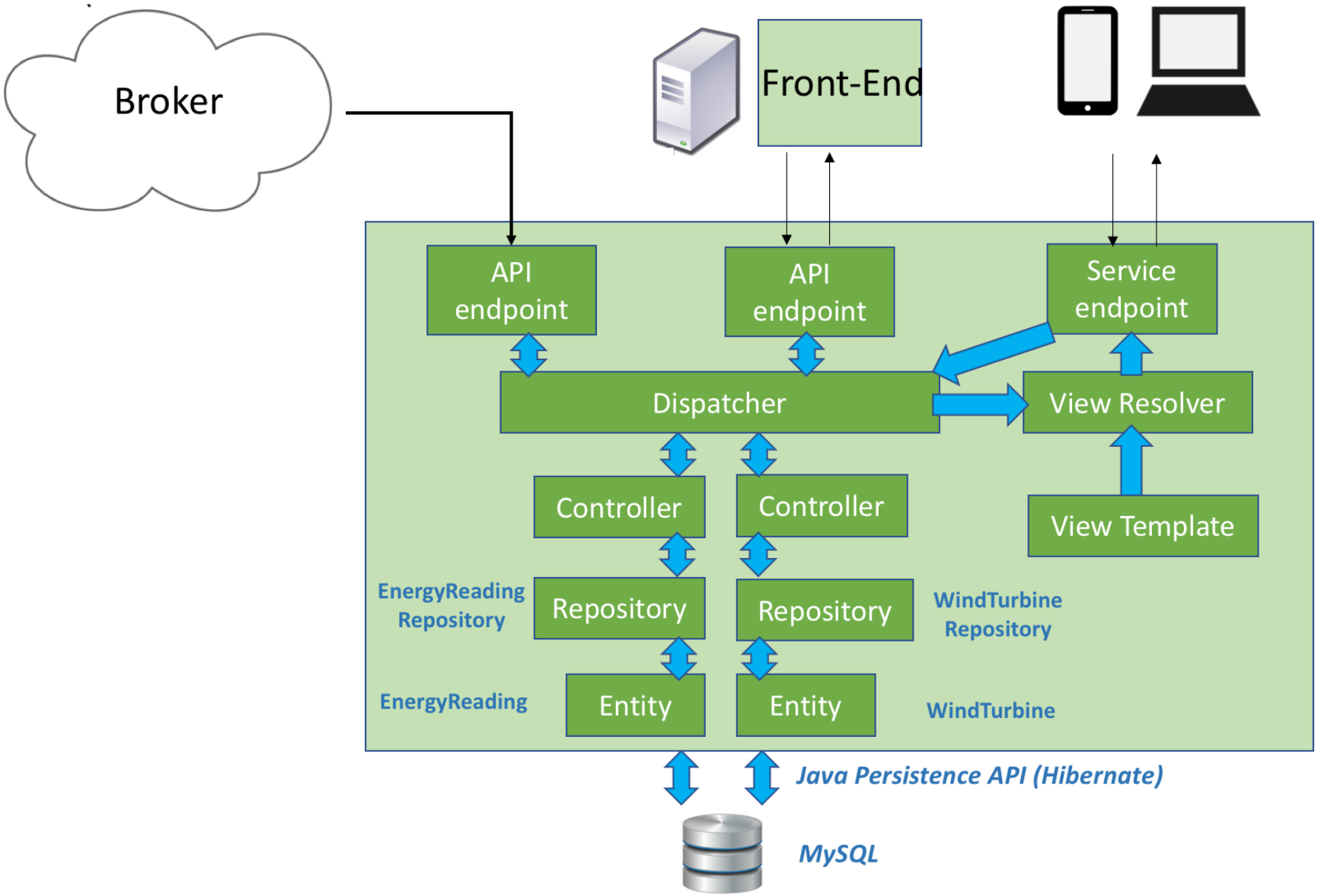
In the Repository Per Entity implementation: create a new Repository implementation for each business object you need to store to or retrieve from your persistence layer.

- Advantage: YAGNI - not implementing methods that are not needed
- Disadvantage: class explosion

Cunostinte avansate despre Repository:

- Common Mistakes with the Repository Pattern
- Why shouldn't I use the Repository Pattern with Entity Framework

Zoom in on server: repositories and entities



Reading Material

Reading Material

- Intro level:
  Design Patterns Explained
  A. Shalloway, Addison Wesley, 2002.
- Professional level:
  Design Patterns: Elements of Reusable Object-Oriented Software.
  E. Gamma, R. Helm, R. Johnson, J. Vlissides, Addison Wesley, 1994.
- Cultura Generala:
  The Timeless Way of Building
  C. Alexander, Oxford University Press, 1977.

De citit

Este util sa cititi textul laboratorului inainte de laborator:

- Lab 10: Design Patterns

- Exemple de folosire pentru Design Patterns in JavaAPI